

DEPI GRADUATION PROJECT

Customer Support RAG-Powered Intelligent Chatbot

Final Graduation Project Report

Retrieval-Augmented Generation • FAISS • FastAPI • Monitoring • Cloud Deployment

Program	DEPI Graduation Project
Project Type	RAG-powered Customer Support Chatbot
Prepared By	Karim Samer Mostafa
Instructor	Omnia Ahmed
Live Deployment	https://support-rag-chatbot-production.up.railway.app
Date	June 2026
Repo	https://github.com/karimsamer100/Customer-Support-RAG-Powered-Intelligent-Chatbot

Table Of Contents

1. Executive Summary 3

2. Problem Statement 3

3. Project Objectives 4

4. Dataset Description..... 4

5. Data Preprocessing and Exploratory Data Analysis 5

 5.1 Data Preprocessing..... 5

 5.2 Exploratory Data Analysis 5

6. RAG Model Development 5

 6.1 Embedding Model 5

 6.2 Vector Store 6

 6.3 Retrieval Pipeline 6

 6.4 Generation and Fallback Handling 6

7. System Architecture..... 7

8. REST API Implementation 8

9. Security Implementation 8

10. Monitoring Dashboard and Feedback 9

11. Evaluation Results 10

12. Cloud Deployment and Integration..... 11

 12.1 Dockerization 11

 12.2 Railway Deployment..... 12

 12.3 Azure App Service Readiness 12

 12.4 Support Portal Integration..... 12

13. MLOps and Re-indexing 13

14. Business KPI Impact..... 14

15. Limitations and Future Work 14

 15.1 Current Limitations 14

 15.2 Future Work 14

16. Conclusion 15

Appendix A: Demo Commands..... 15

Appendix B: Final Deliverables 16

1. Executive Summary

This project presents a cloud-deployed customer support chatbot powered by Retrieval-Augmented Generation (RAG). The system answers customer support questions by retrieving semantically similar historical support cases and using them as grounding context for LLM-based answer generation.

The project combines data preprocessing, exploratory data analysis, vector search, embeddings, LLM generation, REST API deployment, API key security, monitoring, feedback collection, evaluation metrics, Docker deployment, and lightweight MLOps utilities.

The chatbot was successfully deployed as a public cloud service using Railway. The deployed application exposes a public API, health check endpoint, FastAPI documentation, secured chatbot endpoint, feedback endpoint, metrics endpoint, and a monitoring dashboard.

Resource	URL
Public API Root	https://support-rag-chatbot-production.up.railway.app
Health Check	https://support-rag-chatbot-production.up.railway.app/health
Interactive API Docs	https://support-rag-chatbot-production.up.railway.app/docs
Monitoring Dashboard	https://support-rag-chatbot-production.up.railway.app/dashboard

2. Problem Statement

Customer support teams frequently receive repeated questions related to order tracking, refunds, delayed packages, account problems, damaged products, delivery changes, and general support requests. Manual handling of these repeated questions increases response time, support cost, and agent workload.

- Slower first response time for repeated customer questions.
- Higher operational cost due to repetitive manual handling.
- Inconsistent answers when support knowledge is not centralized.
- Reduced customer satisfaction when responses are delayed.
- Limited reuse of historical support solutions.

Traditional chatbots often rely on fixed rule-based flows or the general knowledge of an LLM. This project improves that approach by retrieving relevant historical support cases before generating an answer.

3. Project Objectives

- Collect and preprocess historical customer support data.
- Analyze common support topics using exploratory data analysis.
- Build semantic retrieval using SentenceTransformer embeddings.
- Store and search support knowledge using FAISS.
- Combine retrieval with LLM generation to create a RAG chatbot.
- Expose the chatbot through a FastAPI REST API.
- Secure protected endpoints using API key authentication.
- Add a lightweight monitoring dashboard and metrics endpoint.
- Track request latency, retrieval quality, and feedback.
- Evaluate chatbot performance using retrieval and text similarity metrics.
- Containerize and deploy the system to a public cloud environment.
- Document Azure App Service readiness and future enterprise enhancements.

4. Dataset Description

The project uses historical customer support conversations filtered for AmazonHelp support interactions. The dataset includes customer questions and support responses that are useful for retrieval-based support automation.

File	Description	Status
amazon_final_data.csv	Filtered AmazonHelp support question-answer pairs.	~123k rows
processed_amazon_support.csv	Cleaned and processed support corpus.	~123k rows
vector_store/metadata.csv	Metadata used by the FAISS retrieval pipeline.	20,000 rows
vector_store/faiss_index.bin	FAISS vector index file.	Indexed

Note: The current FAISS index uses a 20,000-row subset of the processed corpus for demo and runtime efficiency. The full 123k corpus is available for complete re-indexing.

5. Data Preprocessing and Exploratory Data Analysis

5.1 Data Preprocessing

The preprocessing pipeline prepares support conversations for semantic embedding and retrieval. The pipeline removes noise and creates clean question-answer pairs that can be used as a searchable knowledge base.

- Load historical support conversations from the raw dataset.
- Filter AmazonHelp interactions from multi-company data.
- Remove noisy text patterns, mentions, and unnecessary tokens.
- Normalize URLs, support links, and repeated whitespace.
- Create cleaned question and answer fields.
- Remove invalid or empty records.
- Save the processed corpus for embedding generation.

5.2 Exploratory Data Analysis

Exploratory Data Analysis was performed to understand frequent question terms, common response terms, issue categories, and repeated support patterns. Results identified dominant themes such as order tracking, refunds, delayed packages, account issues, and product problems.

Deliverable	Status
Processed text corpus	Completed
Preprocessing notebook (DEPI_PROJECT.ipynb)	Completed
EDA notebook and output charts	Completed
Preprocessing documentation	Completed

6. RAG Model Development

Retrieval-Augmented Generation combines information retrieval with text generation. Instead of asking the LLM to answer from general knowledge only, the system first retrieves relevant support examples and passes them to the LLM as context.

6.1 Embedding Model

The project uses the lightweight SentenceTransformer model sentence-transformers/all-MiniLM-L6-v2. This model balances speed and retrieval quality, making it suitable for a demo customer support system without heavy infrastructure requirements.

6.2 Vector Store

The vector store is implemented using FAISS (Facebook AI Similarity Search). FAISS stores vector embeddings and enables fast inner-product similarity search between the user question embedding and the indexed historical support cases.

6.3 Retrieval Pipeline

- Receive and validate the user question.
- Convert the question into a normalized embedding.
- Search the FAISS index for similar support cases.
- Retrieve the top-k most relevant records.
- Filter weak matches using a similarity threshold.
- Remove duplicate retrieved questions.
- Pass the retrieved cases to the generation module.

6.4 Generation and Fallback Handling

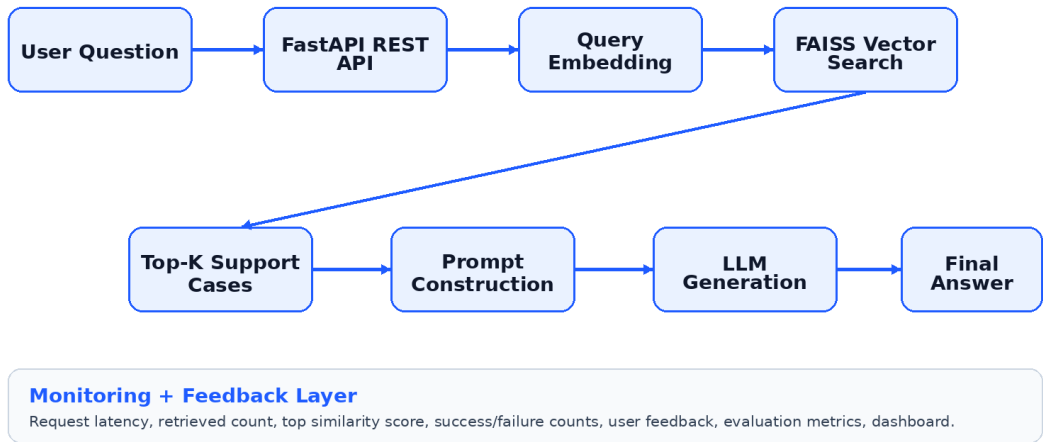
The generation component receives retrieved support cases and builds a prompt for the LLM. If the LLM request fails, the system returns a fallback response based on the most relevant retrieved support case instead of crashing or returning an empty response.

Safety Feature	Purpose
Empty question validation	Prevents invalid API requests.
Top-k clamping	Keeps retrieval size within safe limits.
Similarity threshold	Filters weak or irrelevant matches.
Duplicate removal	Avoids repeated retrieved records.
Fallback response	Keeps the chatbot useful if LLM generation fails.
Pipeline status checks	Improves API reliability and debugging.

7. System Architecture

The system is modular. Each component is implemented in a separate file to support maintainability, testing, and future extension. The flow below shows how a user question travels through the pipeline to produce a grounded answer.

RAG Chatbot System Architecture

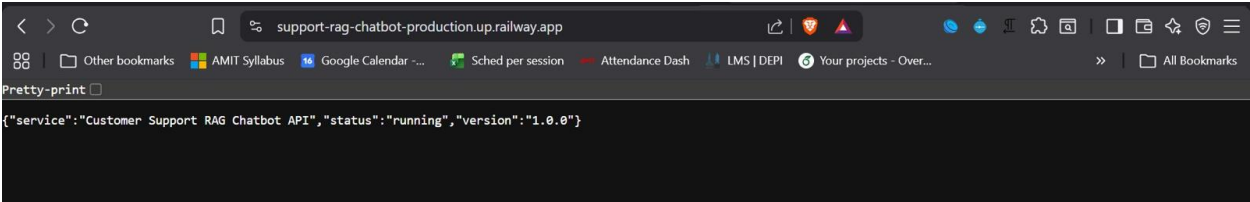


User Question → FastAPI REST API → SentenceTransformer Embedding → FAISS Vector Search → Top-K Retrieved Support Cases → Prompt Construction → LLM Generation → Final Answer + Metadata

Component	File	Purpose
API Service	api.py	Exposes REST endpoints and handles request validation.
Embedding Model	embedder.py	Converts text into normalized vector embeddings.
Index Builder	build_index.py	Builds the FAISS vector index from the corpus.
RAG Pipeline	rag_pipeline.py	Handles retrieval and answer generation workflow.
LLM Generator	llm_generator.py	Calls the LLM provider and handles fallback responses.
Evaluation	evaluate_rag.py	Computes evaluation metrics and generates reports.
Monitoring	monitoring.py	Tracks requests, latency, and feedback logs.
Re-indexing	refresh_index.py	Refreshes the vector index manually.
MLflow Helper	mlflow_tracking.py	Optional experiment tracking helper.

8. REST API Implementation

The backend was implemented using FastAPI. FastAPI provides automatic interactive documentation, fast async support, and a clean structure for REST endpoints. The API is the main interface between external clients and the RAG chatbot pipeline.



Method	Endpoint	Auth	Description
GET	/	No	Root service status.
GET	/health	No	Health check and pipeline status.
POST	/ask	Yes	Ask the chatbot a support question.
POST	/feedback	Yes	Submit user satisfaction feedback.
GET	/metrics	Yes	Return monitoring metrics as JSON.
GET	/dashboard	No	Display monitoring dashboard.
GET	/docs	No	FastAPI interactive API documentation.

Protected endpoints (/ask, /feedback, /metrics) require the header: x-api-key: <APP_API_KEY>

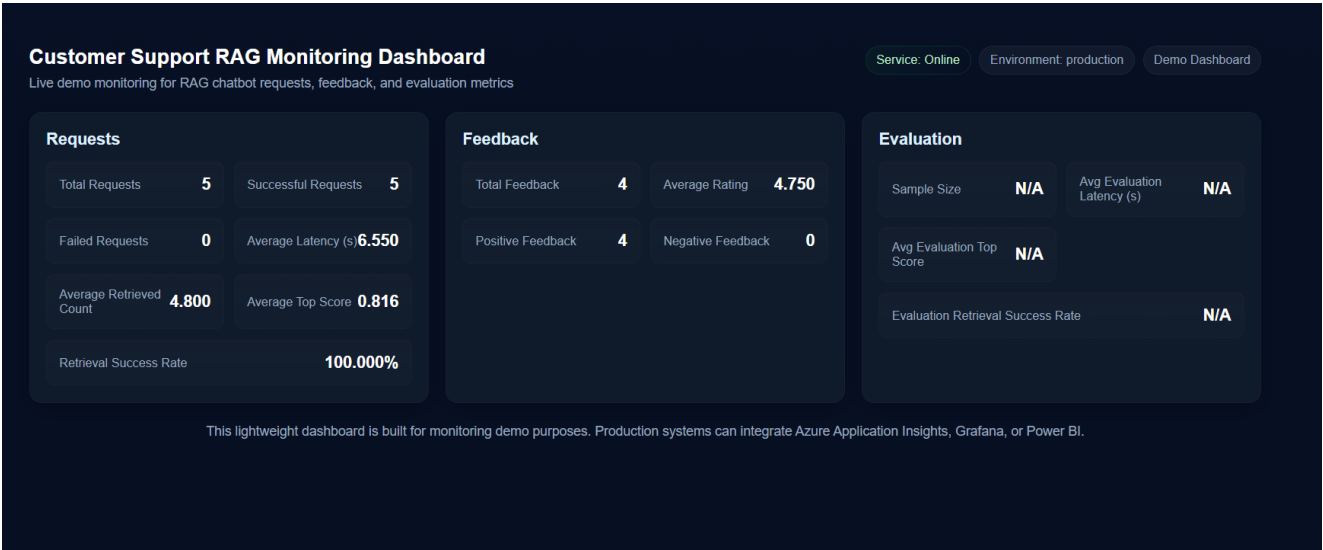
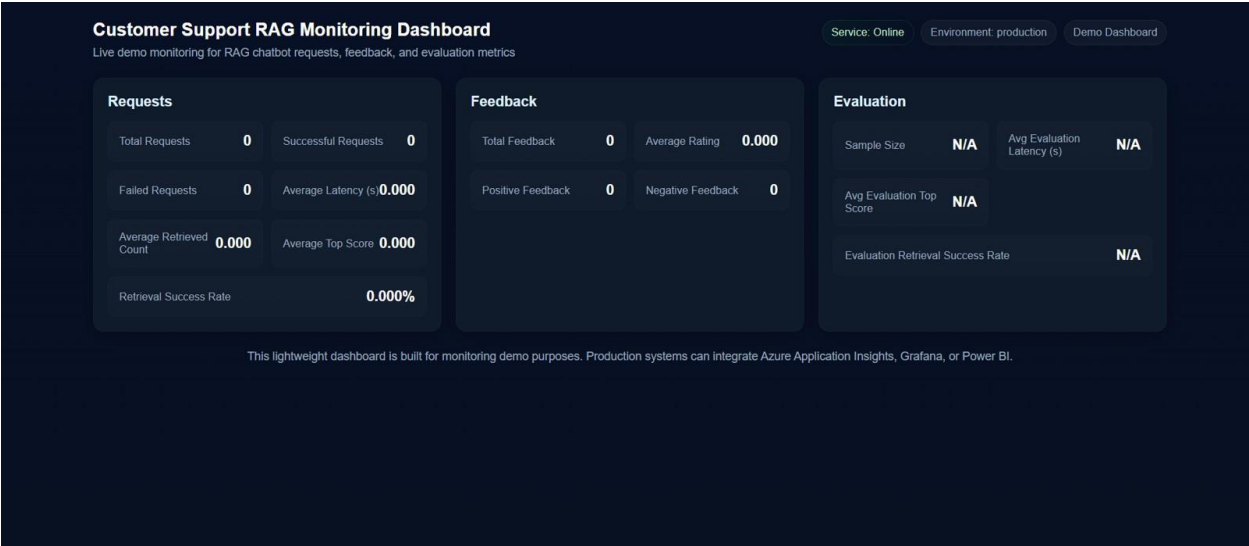
9. Security Implementation

The project implements API key authentication for protected endpoints. API keys are loaded from environment variables and are never hardcoded into the application source code.

- Protected endpoints: /ask, /feedback, and /metrics.
- Secrets are stored in environment variables via a .env file.
- .env is excluded from version control via .gitignore.
- .env.example is provided for safe configuration sharing.
- API key validation is performed through the x-api-key request header.
- Azure AD / Microsoft Entra ID is documented as a future production enhancement.

10. Monitoring Dashboard and Feedback

A lightweight monitoring dashboard was implemented and styled with a clean dark/blue responsive interface suitable for demo screenshots and project presentation. The dashboard automatically refreshes every 30 seconds.



Dashboard Section	Metrics Included	Endpoint
Requests	Total, successful, failed, avg latency, avg retrieved count, avg top score, retrieval success rate.	/dashboard
Feedback	Total feedback, average rating, positive count, negative count.	/dashboard
Evaluation	Sample size, avg evaluation latency, avg top score, evaluation retrieval success rate.	/dashboard

The /feedback endpoint allows users to submit satisfaction ratings (1–5) and optional comments. Feedback is logged in JSONL format and summarized in the dashboard and metrics endpoint.

11. Evaluation Results

Evaluation is implemented using evaluate_rag.py. The script tests retrieval and answer quality using lightweight metrics and saves outputs in the evaluation_results/ directory.

```
python evaluate_rag.py --sample-size 5 --top-k 5
```

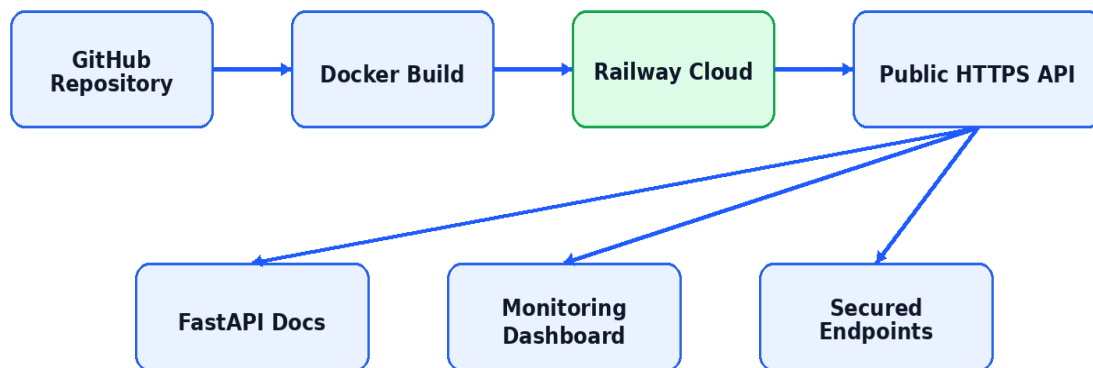
Metric	Value
Sample Size	5
Successful Evaluations	5
Average Latency	0.653 seconds
Average Top Score	1.000
Average Retrieved Count	2.20
BLEU-like Score	0.512
ROUGE-L Score	0.821
Retrieval Success Rate	1.000

The evaluation confirms that the retrieval pipeline is fully functional and consistently retrieves relevant support cases. BLEU-like and ROUGE-L metrics provide a useful sanity check. Production evaluation should also include human relevance scoring and unseen customer questions.

12. Cloud Deployment and Integration

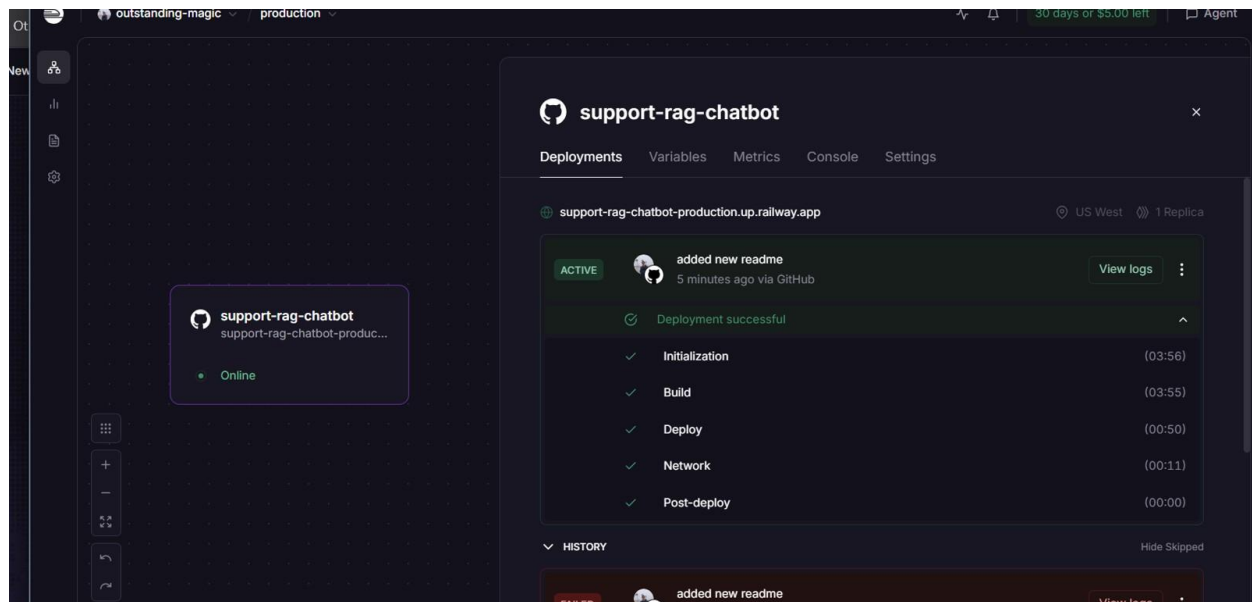
12.1 Dockerization

Cloud Deployment Workflow



Deployed URL: <https://support-rag-chatbot-production.up.railway.app>

Azure App Service migration is documented through Docker-based deployment, environment variables, and /health checks.



The application was containerized using Docker. Docker makes the API portable and allows the same application to run locally, in cloud platforms, or in production environments with consistent dependencies.

12.2 Railway Deployment

The chatbot was successfully deployed as a public cloud service on Railway using Docker, environment variables, a health check endpoint, public HTTPS routing, and API key security.

Deployment Feature	Status
Docker-based build	Implemented
Public HTTPS endpoint	Implemented
Health check endpoint	Implemented
Environment variables	Implemented
API key protected endpoints	Implemented
Monitoring dashboard	Implemented
Feedback endpoint	Implemented
Metrics endpoint	Implemented

12.3 Azure App Service Readiness

Azure App Service deployment documentation is included in docs/azure_deployment_guide.md. Due to Azure subscription constraints, the live demo was hosted on Railway. The same Docker-based deployment pattern can be migrated to Azure App Service by configuring environment variables, the /health check path, and API security settings.

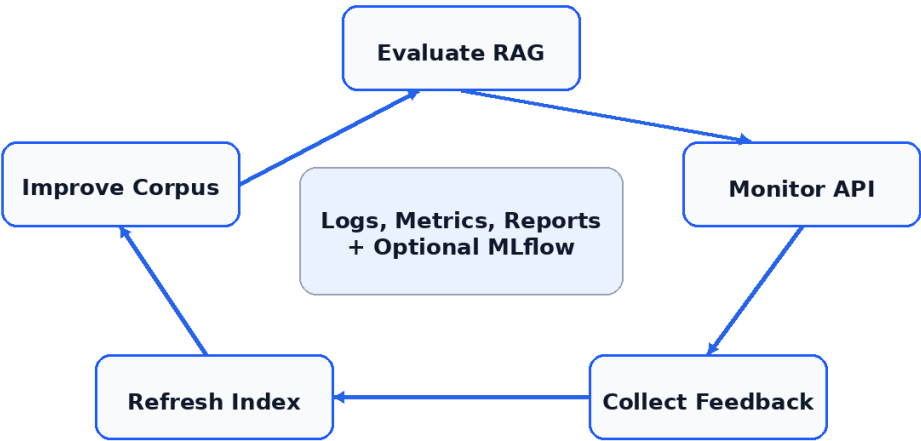
12.4 Support Portal Integration

A simple support portal demo is included in support_portal_demo/index.html. The demo shows how a frontend can call /ask, display the generated answer, show retrieval metadata, and submit feedback to /feedback.

13. MLOps and Re-indexing

The project includes lightweight MLOps support focused on evaluation, request monitoring, feedback tracking, optional experiment logging, and manual re-indexing.

Lightweight MLOps and Monitoring Loop



Current implementation supports manual re-indexing and optional tracking. Scheduling and enterprise observability are future enhancements.

Component	File	Purpose
Evaluation pipeline	evaluate_rag.py	Computes retrieval and generation metrics.
Evaluation outputs	evaluation_results/	Stores metrics, CSV details, and markdown report.
Monitoring helper	monitoring.py	Logs requests and feedback to JSONL files.
Optional MLflow tracking	mlflow_tracking.py	Logs evaluation metrics if MLflow is installed.
Re-indexing script	refresh_index.py	Rebuilds the FAISS vector index manually.

Scheduled retraining is not fully automated in the current implementation. In production, the refresh process can be scheduled using cron, GitHub Actions, Azure Functions, or Azure ML pipelines.

14. Business KPI Impact

The chatbot can improve several business KPIs by reducing repetitive manual support work and increasing response speed and consistency.

KPI	Expected Impact
First Response Time	Faster automated responses for repeated support questions.
Ticket Deflection Rate	Reduces repetitive tickets handled by human agents.
Agent Productivity	Allows agents to focus on complex, non-repetitive issues.
Customer Satisfaction	Provides faster and more consistent support responses.
Support Cost	Reduces operational workload through intelligent automation.
Knowledge Reuse	Reuses historical support solutions effectively at scale.

15. Limitations and Future Work

15.1 Current Limitations

- Knowledge base is limited to historical customer support conversations.
- FAQ, product manuals, and full knowledge base documents are not yet integrated.
- The FAISS index uses a 20,000-row subset of the processed corpus for demo efficiency.
- Evaluation is simplified and should be extended with human relevance scoring.
- Conversation memory is not implemented; responses are single-turn only.
- Azure AD authentication is not implemented; API key auth is used for demo security.
- Scheduled retraining is not fully automated yet; manual re-indexing is provided.
- Logs stored in the demo environment may reset after redeployment.

15.2 Future Work

- Add FAQ, product manuals, and official knowledge base documents to the vector store.
- Add conversation memory for multi-turn support interactions.
- Deploy fully on Azure App Service when a suitable subscription is available.
- Add Azure AD / Microsoft Entra ID authentication for enterprise access control.
- Add CI/CD pipeline for automated testing and deployment.
- Add scheduled embedding refresh and re-indexing with orchestration tools.

- Integrate Azure Application Insights, Grafana, or Power BI for advanced observability.
- Add a human review workflow and more advanced evaluation with unseen questions.
- Integrate with a real customer support ticketing backend.

16. Conclusion

This project successfully demonstrates a complete RAG-powered customer support chatbot workflow. The system starts with historical support data preprocessing, performs semantic embedding generation, builds a FAISS vector index, retrieves relevant support cases, and uses an LLM to generate grounded customer support answers.

The chatbot is exposed through a FastAPI REST API, secured using API key authentication, monitored through a lightweight dashboard, evaluated using retrieval and text similarity metrics, and deployed publicly through Railway using Docker.

Although some enterprise-level features such as Azure AD authentication, fully automated scheduled retraining, and advanced cloud observability are documented as future work, the current implementation provides a strong academic and practical demonstration of RAG, cloud deployment, monitoring, feedback collection, and lightweight MLOps.

Appendix A: Demo Commands

The following PowerShell commands demonstrate the deployed API. Replace <APP_API_KEY> with the configured API key.

Health Check

```
$BASE = "https://support-rag-chatbot-production.up.railway.app"
Invoke-RestMethod -Uri "$BASE/health"
```

Ask Endpoint

```
Invoke-RestMethod -Uri "$BASE/ask" -Method Post `
-headers @{"x-api-key"="<APP_API_KEY>"} `
-ContentType "application/json" `
-Body '{"question":"Where is my order?","top_k":5}'
```

Feedback Endpoint

```
Invoke-RestMethod -Uri "$BASE/feedback" -Method Post `
-headers @{"x-api-key"="<APP_API_KEY>"} `
-ContentType "application/json" `
-Body '{"question":"Where is my order?","answer":"Demo answer","rating":5}'
```

Metrics Endpoint

```
Invoke-RestMethod -Uri "$BASE/metrics" -Headers @{"x-api-key"="<APP_API_KEY>"}
```

Appendix B: Final Deliverables

Deliverable	Status
Processed corpus	Completed
Preprocessing pipeline	Completed
EDA outputs	Completed
FAISS vector store	Completed
RAG chatbot pipeline	Completed
FastAPI REST API	Completed
API key security	Completed
Public cloud deployment (Railway)	Completed
Azure deployment documentation	Completed
Monitoring dashboard	Completed
Feedback endpoint	Completed
Metrics endpoint	Completed
Evaluation report	Completed
MLOps helper scripts	Completed
Support portal demo	Completed
README documentation	Completed
Presentation outline	Completed
Business KPI analysis	Completed